# FitRank AI

Intelligent Candidate Discovery & Ranking — India Runs / Redrob Challenge

## Problem

- Recruiters screening 100K profiles miss strong fits because keyword filters reward resume stuffing, ignore career evidence, and cannot weigh availability signals.
- The Redrob JD explicitly warns against keyword matching — traps include honeypots, consulting-only careers, research-without-production, and inactive candidates.

## Design Philosophy

- Rank like a senior recruiter: understand what the role needs, verify evidence in career history, penalize traps, and down-rank unreachable candidates.
- Hybrid scoring: structural fit (65%) + semantic JD alignment (35%), multiplied by behavioral availability and integrity checks.

## Architecture

- 1. Load candidates.jsonl (100K) in streaming batches.
- 2. Integrity layer flags honeypots (date contradictions, impossible skill durations).
- 3. Structural scorer evaluates title domain, retrieval/ranking career evidence, trusted skills, experience band, evaluation exposure, logistics.
- 4. Semantic scorer uses fast hashing TF vectors (default) or optional MiniLM embeddings.
- 5. Behavioral multiplier uses last_active, response rate, open-to-work, interview completion.
- 6. Top-100 selected with deterministic tie-break (score desc, candidate_id asc).

## Why Not Keywords Alone?

- Career text is weighted higher than skills[] — a Tier-5 engineer who shipped recommenders without buzzwords outranks a Marketing Manager with perfect AI skill lists.
- Skill trust uses proficiency × duration × endorsements — keyword stuffers with zero-duration expert skills score near zero.
- Consulting-only, CV-only, LangChain-wrapper-only, and title-chaser penalties mirror JD disqualifiers.

## Tech Stack

- Python 3.10+, NumPy, scikit-learn HashingVectorizer, optional sentence-transformers.
- CPU-only at ranking time; no external API calls; completes within 5-minute budget on 100K profiles.

## Outputs

- submission.csv — 100 rows: candidate_id, rank, score, reasoning.
- Validated against official validate_submission.py (monotone scores, unique ranks, CAND_XXXXXXX format).

## Reproduction

- pip install -r requirements.txt
- Place candidates.jsonl in data/
- python rank.py --candidates data/candidates.jsonl --out submission.csv --stream
- python data/validate_submission.py submission.csv

## Scoring Formula

| Component | Weight / Role |
|---|---|
| Structural | 65% — title, career evidence, skills trust, experience, evaluation, logistics |
| Semantic | 35% — JD cosine similarity (hashing or embeddings) |
| Behavioral | Multiplier 0.35–1.12 — recency, response rate, open-to-work |
| Integrity | Multiplier — honeypot detection floors bad profiles |
| Penalties | Consulting-only, keyword stuffer, research-only, CV-only, title chaser |

Team: GenAI Bootcamp | Track 1: Intelligent Candidate Discovery